

Keywords: hierarchical robot planning • vision-language models • agentic distillation • stage-transition detection

StageGuard

Three-Layer Agentic Distillation Teaches a Compact VLM to Decide When Robots Should Switch Subtasks—at Real-Time Speed

By Wenbo Cui, Jing Xu, Rui Zhao, Yunchao Liu, Chaobin Ye, Xinyu Zhang & Minghui Tan (South China University of Technology; Tencent Robotics X)

arXiv:2609.20791 • Submitted September 17, 2026

The bottleneck in long-horizon robot planning is not the skills themselves but knowing when to stop one and start the next. *StageGuard* distills structured three-layer reasoning from a large cloud VLM into a compact local model that runs at 11 Hz, achieves F1 = 88.4 on stage-transition detection—7.2 points above the best prior method—and improves full-task success rate by 9.7 percentage points on BEHAVIOR-1K.

AT A GLANCE

Problem: Stage-transition decisions in hierarchical robot planning require human-designed completion checkers that fail under perceptual noise; large VLMs can reason about completion but are misaligned with subtask criteria and too slow for real-time control.

Why it matters: Errors in stage transitions cascade: a premature transition moves to the next skill before the current one is complete, and a missed transition stalls the robot indefinitely.

Method: A teacher VLM generates structured three-layer traces (state, task, decision) from demonstration trajectories; a compact student VLM is fine-tuned on these traces and deployed on-robot.

Result: F1 = 88.4 (sim), 87 ms latency, 67.1% task success rate on BEHAVIOR-1K; validated on a real Franka Panda.

Evidence: Comparison with rule-based checkers, zero-shot GPT-4V, and fine-tuned VLM without structured reasoning; ablations over each reasoning layer and observation history length.

The Big Picture

Hierarchical robot planners decompose complex tasks (e.g., “load the dishwasher and close it”) into a sequence of subtasks (grasp plate, place plate, grasp glass, ..., close door), each executed by a dedicated low-level skill policy. The stage-transition monitor decides when to terminate the current skill and invoke the next.

This is a harder problem than it appears. Low-level skill policies produce smooth trajectories that may overshoot, stall, or partially succeed. The robot may not be able to observe all relevant state directly. Transition criteria that are trivially stated in natural language (“the drawer is fully closed”) require fine-grained spatial reasoning under occlusion and varied lighting.

Current solutions are hand-crafted rule sets: threshold on end-effector position, contact detection, object bounding-box IoU. These rules are brittle, task-specific, and require re-engineering for each new subtask.

Why Existing Approaches Fall Short

Rule-based completion checkers must be designed per subtask by a robotics engineer, do not generalise to novel objects or environments, and fail under sensor noise. Their F1 on BEHAVIOR-1K is only 71.3.

Cloud VLMs (GPT-4V, Qwen-VL-Max) can interpret diverse scene images but have three problems for this use case: (1) their query-answer format is not calibrated to subtask-completion criteria specifically; (2) they operate at 1–3 seconds per query vs. 10 Hz control loops; (3) they require cloud access, making them unsuitable for offline or safety-critical deployments.

Fine-tuned VLMs without structured reasoning achieve F1 = 81.6 but have a flat, one-layer reasoning process: they output binary decisions without decomposing state observations, task semantics, or decision rationales separately, limiting their robustness.

Core Mechanism

The teacher VLM processes each timestep observation through three sequential prompting stages that build a structured trace ($z_{\text{state}}, z_{\text{task}}, z_{\text{decision}}, d_t$):

State layer. Given observation history $o_{t-H:t}$, the teacher extracts a structured state summary: current object poses, grasp contacts, spatial relations, and completion indicators (e.g., “cup rim 2 cm above shelf surface, no contact”).

Task layer. Given z_{state} and the current/next subtask pair (T_i, T_{i+1}) , the teacher maps state facts to subtask completion criteria: which aspects of z_{state} determine whether T_i is done, and what preconditions T_{i+1} requires.

Decision layer. Given z_{task} , the teacher issues a binary transition verdict $d_t \in \{0, 1\}$ with a natural-language rationale z_{decision} .

A compact local student VLM (3B parameters) is fine-tuned on $(o_{t-H:t}, T_i, T_{i+1}) \rightarrow (z_{\text{decision}}, d_t)$ pairs generated by running the teacher on all demonstration trajectories.

Technical Formulation

Formally, the monitor observes $o_t = (I_t, \mathbf{q}_t, \mathbf{p}_t)$ (RGB image, joint configuration, end-effector pose) with history window H . The transition decision:

$$d_t = f(o_{t-H:t}, T_i, T_{i+1}) \in \{0, 1\}.$$

The student produces a soft confidence from its last-layer hidden state:

$$c_t = \sigma(w^\top h_t), \quad h_t = \text{Student}(o_{t-H:t}, T_i, T_{i+1}),$$

triggering a transition when $c_t \geq \tau$ (set by validation).

The student training loss combines binary cross-entropy on the decision and a token-level alignment loss on the compressed explanation:

$$\mathcal{L}_S = \mathcal{L}_{\text{CE}}(d_t, \hat{d}_t) + \lambda \mathcal{L}_{\text{align}}(z_{\text{decision}}, \hat{z}_{\text{decision}}).$$

The alignment loss is a token-level cross-entropy weighted by teacher confidence:

$$\mathcal{L}_{\text{align}} = - \sum_j \hat{p}_j \log p_j^{\text{student}},$$

where $\hat{p}_j$ are teacher token probabilities for explanation token j .

Learning or Inference Procedure

1. Collect human demonstration trajectories for target task set.
2. Run cloud teacher VLM (offline) to produce structured traces for all trajectory timesteps.
3. Fine-tune compact student VLM on trace-decision pairs.
4. Deploy student on-robot: at each 10 Hz control step, pass recent observation and subtask context; trigger transition if $c_t \geq \tau$.
5. Log student explanation text for safety records and debugging.

Training uses $\lambda = 0.3$, history $H = 5$, threshold $\tau = 0.55$. Teacher traces are generated once and cached; student fine-tuning takes approximately 4 hours on a single A100.

What the Guarantee Says

StageGuard provides no formal guarantees. Empirically, the student’s F1 is within 2.3 points of the teacher’s F1 on held-out BEHAVIOR-1K trajectories, while inference latency drops from 1,820 ms (cloud VLM) to 87 ms (local student), enabling real-time monitoring at 11 Hz. Real-robot experiments on five Franka Panda tasks confirm the improvements transfer beyond simulation.

Experimental Findings

Method	F1	Lat. (ms)	Task SR
Rule-based	71.3	2	48.2
GPT-4V zero-shot	79.1	1820	52.7
VLM FT (no reasoning)	81.6	91	57.4
StageGuard	88.4	87	67.1

Task SR = full long-horizon episode success rate on BEHAVIOR-1K simulation. StageGuard improves task success by 9.7 points over the best prior method (VLM FT) at essentially the same inference latency.

Ablations and Interpretation

Removing state layer: F1 drops 4.1 points; spatial relationship reasoning degrades without the structured state summary.

Removing task layer: F1 drops 3.6 points; subtask completion criteria are not grounded without explicit task-plan mapping.

Unstructured distillation: F1 = 83.9 (vs. 88.4); unstructured teacher traces dilute relevant decision factors with spurious reasoning.

History window H : $H = 5$ is optimal; $H = 1$ loses 4.8 F1; $H = 10$ gains 0.4 F1 at double the latency.

Student size: 3B achieves full StageGuard F1; 1B loses 3.2 F1; 7B adds only 0.5 F1 at $3\times$ latency.

Reference

Wenbo Cui, Jing Xu, Rui Zhao, Yunchao Liu, Chaobin Ye, Xinyu Zhang, Minghui Tan. **StageGuard: Learning Stage Transitions for Long-Horizon Robot Tasks via Agentic Distillation**. arXiv:2609.20791, September 2026. <https://arxiv.org/abs/2609.20791>